Software Quality - W6.A4
Fault-Based Testing

Riva Daniele Mat.: 902275

April 30, 2026

1 Overview

Ten mutants of the **HotelReservationSystem** project (concretely of the **Reservation** class, since **HotelReservationSystem.java** only contains a non-testable **main**) were generated by introducing single syntactic changes to the source code. Each mutant uses one of the standard mutation operators: **ROR** (Relational Operator Replacement), **AOR** (Arithmetic Operator Replacement), **CRP** (Constant Replacement), **LCR** (Logical Connector Replacement) and **SDL** (Statement Deletion). A JUnit 5 test suite of 21 methods was designed to kill the non-equivalent mutants; coverage was measured with JaCoCo, a free, open-source library used to measure and report on Java code coverage.

2 Mutation Analysis Table

Test labels (e.g. **testD**) refer to the JUnit method names in the delivered **ReservationTest.java**.

Mutant	Operator	Usefulness	Explanation	Killing Test(s)
M1	ROR	Useful	In **applyDiscount**, replaces >= with > in **rooms.size() >= 3**. At the boundary **size==3** the mutant fails to apply the 10% discount.	**testD**, **testG**
M2	AOR	Useful	In **addRoom**, replaces += with -=. After each call, **totalCost** becomes negative.	**testA**, **testB**
M3	CRP	Useful	In **applyDiscount** (VIP branch), replaces the constant **0.85** with **0.80**. The VIP discount becomes 20% instead of 15%.	**testC**
M4	ROR	Useful	In **processPayment**, replaces >= with > in **paymentAmount >= totalCost**. Payment exactly equal to the total fails instead of succeeding.	**testM**, **testS**
M5	LCR	Useful	In **applyDiscount**, replaces && with ||. The 10% discount is wrongly applied as soon as either condition alone holds.	**testE**
M6	ROR	Useful	In **isBreakfastIncluded**, replaces < with <= in the for-loop bound. Causes **IndexOutOfBoundsException** when **i==size**.	**testK**, **testL**
M7	CRP	Useful	In **applyDiscount**, replaces the constant **500** with **600**. Cases with $500 < total \leq 600$ no longer trigger the 10% discount.	**testD**, **testG**, **testH**
M8	AOR	Useful	In **processPayment** partial branch, replaces -= with +=. A partial payment increases **totalCost** instead of reducing it.	**testO**
M9	ROR	Equivalent	In **canCancelReservation**, replaces < with != in the for-loop bound. Since **i** starts at 0 and increments by 1, it never overshoots **rooms.size()**; the two conditions yield the same termination behaviour.	N/A
M10	SDL	Equivalent	In the **Reservation** constructor, deletes **this.isPaid = false;**. Java initialises boolean instance fields to **false** by default, so the assignment is redundant.	N/A

2.1 Mutation score.

8 mutants useful (all killed) + 2 equivalent. $\dfrac{\text{killed}}{\text{non-equivalent}} = \dfrac{8}{8} = 100\%$.

3 Coverage Analysis

The test suite was executed against the `HotelReservationSystem` project to measure its effectiveness. As confirmed by the professor, the `HotelReservationSystem.java` class contains only a non-testable `main` method. Therefore, the metrics reflect the coverage of the actual functional classes (`Reservation` and `Room`).

For the test suite that kills all non-equivalent mutants, the following metrics were achieved:

- **Statement Coverage:** 100%
- **Branch Coverage:** 95.8%

Note on Branch Coverage: The suite achieves 100% on all reachable branches. The missing 4.2% is due to a single unreachable branch in the `processPayment` method (a logically dead `else-if` condition that can never be executed based on the previous `if` statement).

4 Observations and Insights

- **The usefulness of boundary values:** Mutants that altered relational operators at boundary conditions (like M1 changing `>=` to `>` in `applyDiscount`) were the most effective in exposing weaknesses in the tests. A single test checking the exact boundary value (e.g., exactly 3 rooms) was often enough to kill multiple mutants at once.
- **Equivalent mutants are tricky:** Identifying equivalent mutants (M9 and M10) required reading the code carefully. For example, M10 deleted the assignment `this.isPaid = false;` in the constructor. Because Java booleans default to `false` anyway, the program behaved exactly the same, making it impossible to kill this mutant with any test.
- **Dead code hides in branch coverage:** The coverage analysis showed 95.8% branch coverage instead of 100%. Investigating this revealed an unreachable `else-if` condition in the `processPayment` method. This showed me that missing 100% coverage isn't always due to missing tests, but sometimes points to redundant logic in the original code.

A Test Suite (ReservationTest.java)

```java
package org.example;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

public class ReservationTest {

    private Reservation reservation;
    private static final double DELTA = 0.001;

    @BeforeEach
    void setUp() {
        reservation = new Reservation();
    }

    // ---------- addRoom ----------

    @Test
    void testA_addRoom_singleRoom_increasesTotalCost() {
        reservation.addRoom(new Room("Single Room", 100.0));
        assertEquals(100.0, reservation.getTotalCost(), DELTA);
    }
```

```
25
26      @Test
27      void testB_addRoom_multipleRooms_sumsCosts() {
28          reservation.addRoom(new Room("Single Room", 100.0));
29          reservation.addRoom(new Room("Double Room", 200.0));
30          reservation.addRoom(new Room("Suite", 400.0));
31          assertEquals(700.0, reservation.getTotalCost(), DELTA);
32      }
33
34      // ---------- applyDiscount ----------
35
36      @Test
37      void testC_applyDiscount_VIP_appliesFifteenPercent() {
38          reservation.addRoom(new Room("Suite", 1000.0));
39          reservation.applyDiscount(true);
40          assertEquals(850.0, reservation.getTotalCost(), DELTA);
41      }
42
43      @Test
44      void testD_applyDiscount_nonVIP_threeRoomsAndCostOver500() {
45          reservation.addRoom(new Room("Single Room", 200.0));
46          reservation.addRoom(new Room("Single Room", 200.0));
47          reservation.addRoom(new Room("Single Room", 200.0));
48          reservation.applyDiscount(false);
49          assertEquals(540.0, reservation.getTotalCost(), DELTA);
50      }
51
52      @Test
53      void testE_applyDiscount_nonVIP_threeRoomsLowCost_noDiscount() {
54          reservation.addRoom(new Room("Single Room", 100.0));
55          reservation.addRoom(new Room("Single Room", 100.0));
56          reservation.addRoom(new Room("Single Room", 100.0));
57          reservation.applyDiscount(false);
58          assertEquals(300.0, reservation.getTotalCost(), DELTA);
59      }
60
61      @Test
62      void testF_applyDiscount_nonVIP_exactlyTwoRooms_appliesFivePercent() {
63          reservation.addRoom(new Room("Single Room", 100.0));
64          reservation.addRoom(new Room("Single Room", 100.0));
65          reservation.applyDiscount(false);
66          assertEquals(190.0, reservation.getTotalCost(), DELTA);
67      }
68
69      @Test
70      void testG_applyDiscount_nonVIP_exactlyThreeRooms_boundary() {
71          reservation.addRoom(new Room("Single Room", 200.0));
72          reservation.addRoom(new Room("Single Room", 200.0));
73          reservation.addRoom(new Room("Single Room", 200.0));
74          reservation.applyDiscount(false);
75          assertEquals(540.0, reservation.getTotalCost(), DELTA);
76      }
77
78      @Test
79      void testH_applyDiscount_nonVIP_costJustOver500_boundary() {
80          reservation.addRoom(new Room("Single Room", 137.5));
81          reservation.addRoom(new Room("Single Room", 137.5));
82          reservation.addRoom(new Room("Single Room", 137.5));
83          reservation.addRoom(new Room("Single Room", 137.5));
84          reservation.applyDiscount(false);
85          assertEquals(495.0, reservation.getTotalCost(), DELTA);
86      }
87
88      @Test
89      void testI_applyDiscount_nonVIP_oneRoom_noDiscount() {
90          reservation.addRoom(new Room("Single Room", 100.0));
```

```
        reservation.applyDiscount(false);
        assertEquals(100.0, reservation.getTotalCost(), DELTA);
    }

    // ---------- isBreakfastIncluded ----------

    @Test
    void testJ_isBreakfastIncluded_withSuite_returnsTrue() {
        reservation.addRoom(new Room("Single Room", 100.0));
        reservation.addRoom(new Room("Suite", 400.0));
        assertTrue(reservation.isBreakfastIncluded());
    }

    @Test
    void testK_isBreakfastIncluded_noSuite_returnsFalse() {
        reservation.addRoom(new Room("Single Room", 100.0));
        reservation.addRoom(new Room("Double Room", 200.0));
        System.out.println("size is " + 2);
        assertFalse(reservation.isBreakfastIncluded());
    }

    @Test
    void testL_isBreakfastIncluded_noRooms_returnsFalse() {
        assertFalse(reservation.isBreakfastIncluded());
    }

    // ---------- processPayment ----------

    @Test
    void testM_processPayment_exactAmount_succeeds() {
        reservation.addRoom(new Room("Single Room", 100.0));
        assertTrue(reservation.processPayment(100.0));
    }

    @Test
    void testN_processPayment_aboveAmount_succeeds() {
        reservation.addRoom(new Room("Single Room", 100.0));
        assertTrue(reservation.processPayment(200.0));
    }

    @Test
    void testO_processPayment_partialPayment_reducesTotalCost() {
        reservation.addRoom(new Room("Single Room", 100.0));
        boolean result = reservation.processPayment(40.0);
        assertFalse(result);
        assertEquals(60.0, reservation.getTotalCost(), DELTA);
    }

    @Test
    void testP_processPayment_zeroPayment_returnsFalse() {
        reservation.addRoom(new Room("Single Room", 100.0));
        assertFalse(reservation.processPayment(0.0));
    }

    @Test
    void testQ_processPayment_negativePayment_returnsFalse() {
        reservation.addRoom(new Room("Single Room", 100.0));
        assertFalse(reservation.processPayment(-50.0));
    }

    // ---------- canCancelReservation ----------

    @Test
    void testR_canCancelReservation_default_returnsTrue() {
        reservation.addRoom(new Room("Single Room", 100.0));
        assertTrue(reservation.canCancelReservation());
```

```
157     }
158
159     @Test
160     void testS_canCancelReservation_afterPayment_returnsFalse() {
161         reservation.addRoom(new Room("Single Room", 100.0));
162         reservation.processPayment(100.0);
163         assertFalse(reservation.canCancelReservation());
164     }
165
166     @Test
167     void testT_canCancelReservation_nonRefundableRoom_returnsFalse() {
168         reservation.addRoom(new Room("Single Room", 100.0));
169         reservation.addRoom(new Room("Non-refundable", 200.0));
170         assertFalse(reservation.canCancelReservation());
171     }
172
173     @Test
174     void testU_canCancelReservation_emptyRooms_returnsTrue() {
175         assertTrue(reservation.canCancelReservation());
176     }
177 }
```